
Dossier technique de soutenance

Application de gestion des agences bancaires et volet décisionnel Power BI

Chaque fonctionnalité de l'application, sa mise en œuvre côté présentation, contrôle, métier et persistance, puis la chaîne décisionnelle

Projet	Système intelligent de gestion des rôles et de pointage avec tableau de bord décisionnel
Organisme d'accueil	BTK — Banque Tuniso-Koweïtienne
Étudiant	Oussema Korbosli
Parcours	Licence Business Computing — Business Intelligence
Établissement	Esprit School of Business
Année	2025 / 2026

Ce dossier est un **support de préparation à la soutenance technique**. Tout ce qu'il contient est relevé dans le code du projet : servlets, services, entités, pages JSP, scripts SQL, chaîne ETL et fichiers Power BI. Les chiffres cités (18 servlets, 16 services, 16 entités, 17 pages JSP) sont issus d'un inventaire automatique du dépôt.

Table des matières

1	Vue d'ensemble	3
1.1	La pile technique	3
1.2	Ce que contient le projet	3
1.3	L'architecture en quatre couches	3
1.4	Le cycle de vie d'une requête, sur un exemple	4
1.5	Pourquoi les JSP sont sous WEB-INF	4
2	Le socle transverse	4
2.1	Authentification et rôles	5
2.2	Le filtre de notifications	5
2.3	Le journal d'audit	6
2.4	Les notifications	6
2.5	Le gabarit de page	6
2.6	Les seize entités et leurs tables	7
3	Les fonctionnalités, une par une	7
3.1	Authentification	7
3.2	Tableau de bord	8
3.3	Annuaire des employés	8
3.4	Portefeuille clients	8
3.5	Référentiel des agences	8
3.6	Objectifs commerciaux	9
3.7	Pointage de la présence	9
3.8	Demandes de création	9
3.9	Demandes de modification — la double validation	10
3.10	Ajout d'un employé	10
4	Le front en détail	11
4.1	Rien de construit, tout d'écrit	11
4.2	La feuille de style	11
4.3	JSTL et le langage d'expression	11
4.4	Les trois graphiques du tableau de bord	11
5	Le back en détail	11
5.1	Le motif commun à tous les servlets	12
5.2	Les EJB	12
5.3	JPA et les transactions	12
5.4	L'unité de persistance	12
6	Le volet décisionnel	13
6.1	Les vues décisionnelles	13
6.2	Le schéma BTK_BI	13
6.3	La chaîne ETL	14
6.4	Power BI — la construction du rapport	14
6.4.1	Connexion	14
6.4.2	Thème et gabarit	14
6.4.3	Les quatre pages	15
6.4.4	Les mesures DAX	15
6.5	La segmentation des agences	15

7	Construction et déploiement	16
7.1	De la source au WAR	16
7.2	Ce que contient le WAR	16
7.3	Déployer sur WildFly	16
7.4	La datasource	16
7.5	Diagnostiquer une erreur de connexion	17
7.6	L'ordre de démarrage, avant la soutenance	17
8	Les questions probables, et quoi répondre	17
8.1	Architecture	17
8.2	Sécurité	18
8.3	JPA et transactions	18
8.4	Front	18
8.5	Décisionnel	18
9	Points ouverts et limites	19
10	Récapitulatif : la carte des URL	19

1 Vue d'ensemble

1.1 La pile technique

Couche	Technologie retenue
Présentation	JSP + JSTL, CSS écrit à la main (css/style.css, 794 lignes), graphiques Apex-Charts, icônes Lucide
Contrôle	Servlets Jakarta annotées @WebServlet, plus un filtre @WebFilter("/")
Métier	EJB @Stateless (et un @Singleton pour la sécurité), injectés par @EJB
Persistance	JPA / Hibernate, @PersistenceContext, requêtes JPQL
Transactions	JTA, gérées par le conteneur (CMT) — aucun commit écrit à la main
Base de données	Oracle, accédée par la datasource java:/OracleDS
Serveur	WildFly, racine de contexte /gestion-agences
Construction	Maven, empaquetage war, JDK 21
Décisionnel	Vues V_BI_* en Oracle, chaîne ETL Python, Power BI Desktop

En une phrase. Une application web Jakarta EE 10 en couches, servie par WildFly, qui lit et écrit dans Oracle par JPA, et dont les mêmes données alimentent un tableau de bord embarqué et un rapport Power BI par l'intermédiaire de vues dédiées et d'une chaîne ETL.

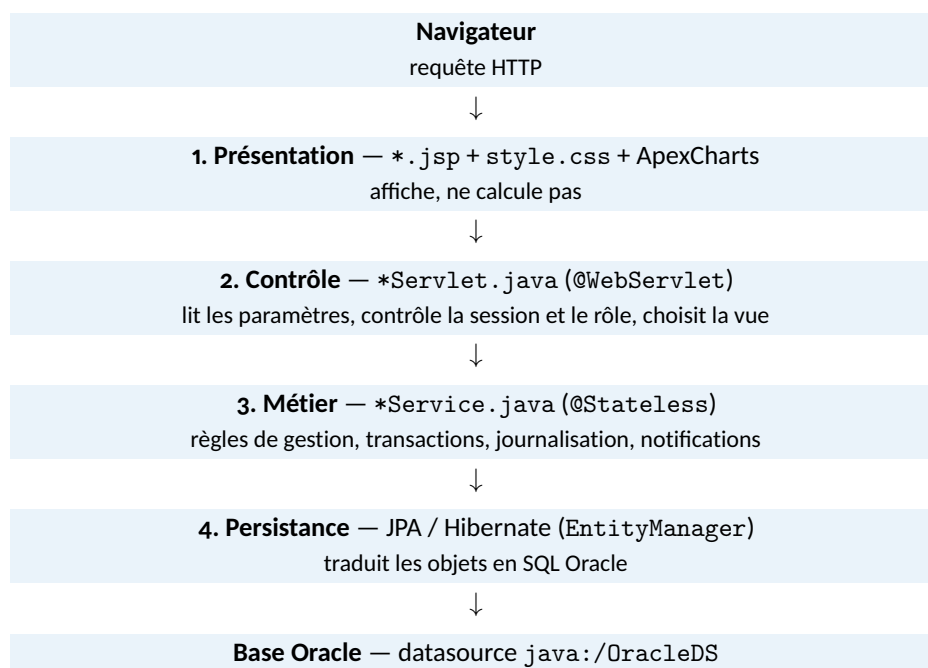
1.2 Ce que contient le projet

Inventaire relevé dans le dépôt :

Élément	Nombre	Emplacement
Servlets	18	src/main/java/controller/
Services EJB	16	src/main/java/service/
Entités JPA	16	src/main/java/entity/
Filtre	1	src/main/java/filter/NotificationFilter.java
Pages JSP	17	src/main/webapp/ et WEB-INF/
Feuille de style	1	src/main/webapp/css/style.css
Vues décisionnelles	4	sql/v_bi_*.sql, sql/setup_pointage.sql

1.3 L'architecture en quatre couches

Chaque requête traverse les mêmes quatre couches, dans le même ordre. C'est le point que le jury vérifie en premier : **aucune couche n'en saute une autre**. Un servlet ne parle jamais à la base ; un service ne fabrique jamais de HTML.



1.4 Le cycle de vie d'une requête, sur un exemple

Prenons l'affichage de la liste des agences, /agences. C'est le parcours à savoir raconter de mémoire.

1. Le navigateur demande GET /gestion-agences/agences.
2. Le **filtre** NotificationFilter s'exécute d'abord : il lit le rôle en session et pose dans la requête les compteurs de demandes en attente, pour que le bandeau les affiche.
3. WildFly route vers AgenceServlet, associé à /agences par l'annotation @WebServlet.
4. Le servlet vérifie la **session** et le **rôle**. Sans session valide, redirection vers la page de connexion.
5. Il appelle agenceService.findAll(). L'EJB ouvre une transaction JTA, exécute une requête JPQL, Hibernate la traduit en SQL, Oracle répond.
6. Le servlet range le résultat dans la requête (setAttribute) et **transfère** vers /WEB-INF/agences.jsp.
7. La JSP parcourt la liste avec <c:forEach> et produit le HTML, qui part au navigateur.

Transfert ou redirection ? Un forward reste côté serveur : l'URL ne change pas et les attributs de requête survivent — c'est ce qu'on utilise pour *afficher*. Un sendRedirect demande au navigateur de refaire une requête : l'URL change et les attributs sont perdus — c'est ce qu'on utilise *après une écriture*, pour qu'un rafraîchissement ne rejoue pas l'enregistrement.

1.5 Pourquoi les JSP sont sous WEB-INF

Toutes les vues sauf deux sont dans src/main/webapp/WEB-INF/. Le conteneur **interdit** d'y accéder par une URL directe : on ne peut y entrer que par un forward depuis un servlet. Conséquence, aucune page ne peut être ouverte en contournant le contrôle de session et de rôle.

Les deux exceptions sont volontaires : index.jsp (page de connexion) et forgot-password.jsp, qui doivent être atteignables sans être authentifié.

2 Le socle transverse

Cinq mécanismes servent à toutes les fonctionnalités. Les connaître évite de répéter la même explication douze fois pendant la soutenance.

2.1 Authentification et rôles

L'authentification s'appuie sur la table `SECURITY_USERS`, portée par l'entité `SecurityUser` et interrogée par `SecurityService`. Ce service est le seul `@Singleton` du projet, et il est marqué `@Startup` : une instance unique, créée au démarrage du serveur.

```
@Singleton
@Startup
public class SecurityService {
    @PersistenceContext
    private EntityManager em;

    public SecurityUser authenticate(String username, String password) {
        TypedQuery<SecurityUser> query = em.createQuery(
            "SELECT s FROM SecurityUser s WHERE s.username = :username "
            + "AND s.password = :password", SecurityUser.class);
        query.setParameter("username", username);
        query.setParameter("password", password);
        List<SecurityUser> results = query.getResultList();
        return results.isEmpty() ? null : results.get(0);
    }
}
```

Le `LoginServlet` appelle ce service, puis, en cas de succès, place trois attributs en session — et rien d'autre :

```
HttpSession session = request.getSession();
session.setAttribute("user", username);
session.setAttribute("role", role);
session.setAttribute("userId", businessUser.getId());
response.sendRedirect(request.getContextPath() + "/home");
```

Trois rôles circulent dans l'application, exactement ceux stockés dans `SECURITY_USERS` :

Rôle	Périmètre
ADMIN	Tout le périmètre fonctionnel
DIRECTEUR_COMMERCIAL	Soumet les demandes de création, valide les demandes de modification, consulte
USER	Son pointage, ses demandes, ses notifications

Le contrôle est fait **dans chaque servlet**, au début de `doGet` et de `doPost`, selon un motif constant :

```
HttpSession session = request.getSession(false);
if (session == null
    || session.getAttribute("user") == null
    || !"ADMIN".equals(session.getAttribute("role"))) {
    response.sendRedirect(request.getContextPath() + "/index.jsp");
    return;
}
```

Pourquoi `getSession(false)` ? Avec `true` (le défaut), l'appel *crée* une session si elle n'existe pas — le test « la session est-elle nulle » ne détecterait alors plus jamais un visiteur non connecté. Avec `false`, la méthode renvoie `null` quand aucune session n'existe : c'est ce qui permet le contrôle.

2.2 Le filtre de notifications

`NotificationFilter` est annoté `@WebFilter("/*")` : il s'exécute **avant tout servlet**, sur toutes les URL. Son rôle est d'alimenter les compteurs affichés dans le bandeau, sans que chaque servlet ait à le refaire.

```
if (session != null) {
    String role = (String) session.getAttribute("role");
    if ("ADMIN".equals(role)) {
```

```

        request.setAttribute("pendingClientDemandes", demandeClientService.countPending());
        request.setAttribute("pendingEmployeDemandes", demandeEmployeService.countEnAttente());
        request.setAttribute("approvedModifs", demandeModificationService.countInCours());
    } else if ("DIRECTEUR_COMMERCIAL".equals(role)) {
        request.setAttribute("pendingModifs", demandeModificationService.countPending());
    }
}
chain.doFilter(request, response);

```

Les compteurs dépendent du rôle : l'administrateur voit ce qu'il doit *exécuter*, le directeur ce qu'il doit *valider*.

2.3 Le journal d'audit

JournalService écrit dans JOURNAL_ACTION (entité JournalAction). Chaque écriture sensible appelle `log(...)` avec l'auteur, son rôle, l'action, le type et l'identifiant de l'objet touché, et un libellé. Exemple relevé dans `DemandeAgenceAdminServlet` :

```

journalService.log(userId, role,
    "APPROVE_DEMANDE_AGENCE", "DEMANDE_AGENCE", demande.getId(),
    "Approbation de la demande d'agence code=" + demande.getCodeAgencePropose());

```

2.4 Les notifications

NotificationService écrit dans NOTIFICATION. Une notification vise **un utilisateur** et porte un type et un message :

```

notificationService.notifyUser(demande.getDirecteur(),
    "DEMANDE_AGENCE_VALIDEE",
    "Votre demande d'agence " + demande.getCodeAgencePropose() + " a ete approuvee.");

```

C'est le mécanisme qui referme chaque circuit de demande : celui qui a soumis est prévenu de la décision.

2.5 Le gabarit de page

Deux fragments sont inclus par toutes les pages authentifiées :

- `WEB-INF/sidebar.jsp` — le menu latéral. Les entrées sont conditionnées par le rôle avec `<c:if>`; le paramètre `activePage` met en relief l'entrée courante.
- `WEB-INF/top-bar.jsp` — le bandeau, qui affiche les compteurs posés par le filtre et oriente le clic selon le rôle :

```

<c:set var="notifLink"
    value="${sessionScope.role == 'ADMIN' ? 'demandes-admin' : 'gestion-demandes'}" />

```

2.6 Les seize entités et leurs tables

Entité	Table Oracle	Ce qu'elle porte
SecurityUser	SECURITY_USERS	comptes et rôles
Utilisateur	B_UTILISATEURS	employés (dont les gestionnaires)
Agence	AGENCE	référentiel des agences
Banque	B_BANQUE	entités bancaires
BClient	B_CLIENTS	clients enrichis (secteur, région)
BtkClient	CLIENT_BTK	clients du réseau
Client	CLIENT	table client historique
Gestionnaire	GESTIONNAIRE	axe gestionnaire
Objectif	B_OBJECTIF	production commerciale par période
Pointage	POINTAGE	présence : arrivée, départ, statut
DemandeAgence	DEMANDE_AGENCE	demandes de création d'agence
DemandeEmploye	DEMANDE_EMPLOYE	demandes de création d'employé
DemandeClient	DEMANDE_CLIENT	demandes de création de client
DemandeModification	DEMANDES_MODIFICATION	demandes de modification de profil
Notification	NOTIFICATION	messages destinés à un utilisateur
JournalAction	JOURNAL_ACTION	journal d'audit

3 Les fonctionnalités, une par une

Chaque fiche suit le même plan : ce que la fonctionnalité fait, ce qui se passe côté **front**, ce qui se passe côté **back**, et les données touchées. Les fiches sont dans l'ordre du parcours d'un utilisateur.

3.1 Authentification

URL	/login, /logout, /forgot-password
Rôles	tous (pages ouvertes)

Ce que ça fait. Contrôle l'identifiant et le mot de passe contre la table SECURITY_USERS, ouvre une session et oriente vers le tableau de bord.

Front. index.jsp — la seule page hors de WEB-INF avec forgot-password.jsp, puisqu'il faut pouvoir l'atteindre sans être connecté. Formulaire en POST, message d'erreur affiché depuis l'attribut error.

Back. LoginServlet : doGet transfère vers index.jsp; doPost appelle securityService.authenticate(...), puis utilisateurService.findOrCreate(...) pour rattacher le compte de sécurité à l'employé métier de B_UTILISATEURS. Les trois attributs de session sont posés, et une **redirection** envoie vers /home. LogoutServlet appelle session.invalidate().

Données. SECURITY_USERS (lecture), B_UTILISATEURS (lecture, création si le compte métier n'existe pas encore).

Limite assumée, à annoncer avant qu'on vous la pose. Le mot de passe est comparé **en clair** dans la requête JPQL. En production il faudrait un condensat salé (BCrypt ou Argon2) et une comparaison en Java, pas en SQL. Le choix a été fait pour rester dans le périmètre du stage ; la correction est localisée dans une seule méthode, SecurityService.authenticate.

3.2 Tableau de bord

URL	/home
Rôles	tous les utilisateurs connectés

Ce que ça fait. Page d'accueil après connexion : cartes d'indicateurs, trois graphiques, et les « actions requises » propres au rôle.

Front. `home.jsp`. Trois graphiques **ApexCharts** construits en JavaScript à partir de valeurs injectées par le servlet : `#agencyChart` (histogramme des effectifs par agence), `#roleChart` (anneau de la répartition par rôle) et `#pointageChart` (anneau présents / retards / absents). Les blocs « actions requises » sont conditionnés par `<c:if>` sur le rôle.

Back. `HomeController` n'a qu'un `doGet`, mais il agrège **sept services** pour composer la page : effectifs et répartition par rôle, portefeuille clients, demandes en attente, et la présence du jour — calculée à partir de `countByStatutBetween` pour chacun des trois statuts, dont il déduit le taux de présence.

Données. `B_UTILISATEURS`, `CLIENT_BTK`, `POINTAGE`, les trois tables de demandes.

Question classique : pourquoi le tableau de bord ne fait-il pas de requête SQL ? Parce que chaque chiffre vient d'une méthode de service (`countTotal`, `countGestionnaires`, `countPresentToday...`). Le servlet ne fait qu'assembler. C'est ce qui permet de réutiliser les mêmes compteurs dans le filtre de notifications sans les réécrire.

3.3 Annuaire des employés

URL	/employes
Rôles	tout utilisateur connecté

Ce que ça fait. Liste des employés de `B_UTILISATEURS` avec leur rôle calculé (gestionnaire, conseiller, hors commercial) et leur agence.

Front. `employes.jsp` : tableau parcouru par `<c:forEach>`, avec recherche et filtre par agence.

Back. `BUtilisateurServlet`, un seul `doGet`, appuyé sur `UtilisateurService` — le service le plus fourni du projet (16 méthodes : `findGestionnaires`, `findConseillers`, `findHorsCommercial`, `findByAgence...`).

Données. `B_UTILISATEURS`, jointe à `AGENCE`.

3.4 Portefeuille clients

URL	/clients-btk
Rôles	ADMIN

Ce que ça fait. Consultation du portefeuille et **soumission d'une demande de création de client** par le directeur commercial.

Front. `clients-btk.jsp` : liste paginée, recherche, formulaire de création.

Back. `BClientServlet` mobilise six services : `BClientService` pour la lecture, `DemandeClientService` pour le circuit de validation, plus `Notification`, `Journal`, `Security` et `Utilisateur`.

Données. `B_CLIENTS`, `CLIENT_BTK`, `DEMANDE_CLIENT`.

3.5 Référentiel des agences

URL	/banques (consultation), /agences (administration)
Rôles	ADMIN

Ce que ça fait. Deux écrans distincts : `/banques` présente le réseau et les employés d'une agence ; `/agences` est le CRUD complet du référentiel.

Front. `banques.jsp` et `agences.jsp`. Dans le menu, seul `/agences` est réservé à l'administrateur par un `<c:if>`.

Back. BanqueServlet (7 méthodes de service, dont `findUtilisateursByAgence`) et AgenceServlet, qui expose les cinq opérations `findAll`, `findById`, `create`, `update`, `delete` et journalise chaque écriture.

Données. AGENCE, B_BANQUE, B_UTILISATEURS.

3.6 Objectifs commerciaux

URL	/objectifs-btk
Rôles	tout utilisateur connecté

Ce que ça fait. Affiche la production commerciale par agence et par gestionnaire : ouvertures de comptes, crédits, épargne.

Front. `objectifs-btk.jsp`, tableau filtrable.

Back. BObjectifServlet et BObjectifService (`findAll`, `findByAgence`, `findByGestionnaire`, `countTotal`).

Données. B_OBJECTIF, dont les colonnes servent aussi de socle à la vue V_BI_OBJECTIFS et aux mesures DAX de Power BI.

3.7 Pointage de la présence

URL	/pointage
Rôles	ADMIN pour la saisie manuelle ; tout employé pour son propre pointage

Ce que ça fait. L'employé enregistre son arrivée puis son départ d'un clic ; l'administrateur peut saisir ou corriger un pointage.

Front. `pointage.jsp` : deux boutons (arrivée, départ), la liste des cinquante derniers pointages, et un formulaire de saisie manuelle visible seulement pour l'administrateur.

Back. PointageServlet route sur le paramètre `action` : `pointerArrivee`, `pointerDepart` ou `save-Manuel`. Chaque action est journalisée, puis le servlet **redirige** vers `/pointage`.

La règle métier est dans le service, pas dans le servlet :

```
private static final LocalTime HEURE_LIMITE = LocalTime.of(9, 0);
...
p.setStatut(now.toLocalTime().isAfter(HEURE_LIMITE) ? "RETARD" : "PRESENT");
```

Le champ SOURCE distingue AUTO (bouton) de MANUEL (saisie administrateur) : la traçabilité de la correction est conservée.

Données. POINTAGE (HEURE_ARRIVEE, HEURE_DEPART, STATUT, SOURCE, COMMENTAIRE), JOURNAL_ACTION.

Trois garde-fous à citer. Un deuxième clic sur « arrivée » n'écrase pas l'heure déjà enregistrée ; un départ sans arrivée préalable ne crée rien ; le statut est calculé par le serveur, jamais envoyé par le formulaire.

3.8 Demandes de création

URL	/demandes-agence puis /demandes-admin
Rôles	soumission : DIRECTEUR_COMMERCIAL ; validation : ADMIN

Ce que ça fait. Le directeur commercial demande la création d'une agence, d'un employé ou d'un client ; l'administrateur approuve ou refuse. À l'approbation, l'objet est **réellement créé**.

Front. `demandes-agence.jsp` côté soumission, `demandes-admin.jsp` côté validation, avec un bloc par type de demande.

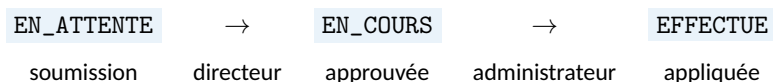
Back. DemandeAgenceServlet crée la demande, notifie son auteur, puis notifie **tous les administrateurs** (`securityService.findAllAdmins()`). DemandeAgenceAdminServlet traite trois circuits dans le même `doPost`, distingués par le nom du paramètre : `action` (agence), `actionEmp` (employé), `actionCli` (client). À l'approbation d'un employé, le servlet construit et enregistre l'objet Utilisateur ; à celle d'un client, un BClient avec le statut ACTIF.

Données. DEMANDE_AGENCE, DEMANDE_EMPLOYE, DEMANDE_CLIENT, puis B_UTILISATEURS / B_CLIENTS, NOTIFICATION, JOURNAL_ACTION.

3.9 Demandes de modification — la double validation

URL	/mes-demandes puis /gestion-demandes
Rôles	soumission : tout employé; approbation : DIRECTEUR_COMMERCIAL; exécution : ADMIN

Ce que ça fait. C'est le circuit le plus intéressant à présenter : la modification d'une donnée personnelle passe par **deux validations successives**, portées par deux rôles différents.



Le directeur peut aussi refuser : l'état devient REFUSE et le circuit s'arrête.

Front. profil.jsp pour soumettre; gestion-demandes.jsp pour traiter. Les boutons sont conditionnés par le rôle et par l'état, ce qui garantit qu'une demande ne peut pas être approuvée deux fois :

```

<c:if test="${sessionScope.role == 'DIRECTEUR_COMMERCIAL' && d.statut == 'EN_ATTENTE'}">
  ... Approuver / Refuser ...
</c:if>
<c:if test="${sessionScope.role == 'ADMIN' && d.statut == 'EN_COURS'}">
  ... Appliquer le changement ...
</c:if>

```

Back. GestionModificationsServlet croise l'action et le rôle, et refuse toute combinaison non prévue :

```

if ("approve".equals(action) && "DIRECTEUR_COMMERCIAL".equals(role)) {
    demandeModificationService.approveRequest(id, comment);
} else if ("reject".equals(action) && "DIRECTEUR_COMMERCIAL".equals(role)) {
    demandeModificationService.rejectRequest(id, comment);
} else if ("execute".equals(action) && "ADMIN".equals(role)) {
    demandeModificationService.executeRequest(id);
} else {
    throw new Exception("Action non autorisee pour votre role.");
}

```

Le service ajoute une seconde barrière : chaque méthode vérifie l'état de départ avant d'agir (approveRequest n'accepte que EN_ATTENTE, executeRequest que EN_COURS). C'est seulement à l'exécution que la donnée est réellement modifiée dans B_UTILISATEURS.

Données. DEMANDES_MODIFICATION, puis B_UTILISATEURS, NOTIFICATION, JOURNAL_ACTION.

Le contrôle est fait à trois endroits — affichage du bouton (JSP), croisement action/rôle (servlet), état de la demande (service). Si on demande « et si quelqu'un forge la requête HTTP ? », la réponse est : la vue est contournable, les deux autres ne le sont pas.

3.10 Ajout d'un employé

URL	/ajouter-employe
Rôles	ADMIN

Ce que ça fait. Création directe d'un employé par l'administrateur, avec son compte de connexion.

Front. add-effectif.jsp, formulaire de saisie.

Back. AjouterEmployeServlet coordonne six services : il crée l'Utilisateur métier, le SecurityUser associé, notifie et journalise.

Données. B_UTILISATEURS, SECURITY_USERS, DEMANDE_EMPLOYE.

4 Le front en détail

4.1 Rien de construit, tout d'écrit

Le projet n'utilise **aucun framework front** : pas de React, pas de Bootstrap, pas d'étape de compilation JavaScript. Le HTML est produit par le serveur, la mise en forme tient dans une feuille de style écrite à la main, et deux bibliothèques seulement sont chargées depuis un CDN :

ApexCharts	cdn.jsdelivr.net/npm/apexcharts — les trois graphiques du tableau de bord
Lucide	unpkg.com/lucide@latest — les icônes (<i data-lucide="...">)

Pourquoi ce choix ? Une application interne, rendue côté serveur, n'a pas besoin d'un client riche : les pages sont des listes et des formulaires. Ne pas ajouter de *framework*, c'est aussi ne pas ajouter d'étape de construction, de gestionnaire de paquets ni de dépendances à maintenir. La contrepartie est qu'il n'y a pas de composants réutilisables : le facteur commun est obtenu par les inclusions JSP (sidebar, top-bar) et par les classes CSS.

4.2 La feuille de style

css/style.css, 794 lignes, organisée autour de **variables CSS** déclarées sur :root — couleurs de la charte BTK, fonds, texte, états. Changer la charte se fait en un endroit. Les classes principales sont card, stat-card-modern, nav-item, btn, form-control, table.

4.3 JSTL et le langage d'expression

Les vues n'embarquent **aucun code Java** : pas de scriptlet <% ... %>. Tout passe par JSTL et le langage d'expression, ce qui garde la vue lisible et sans logique métier.

Balise	Usage dans le projet
<c:if test="\${...}">	afficher un bloc selon le rôle ou l'état
<c:forEach var items>	parcourir une liste posée par le servlet
<c:choose> <c:when>	alternative à plusieurs branches
<c:set var value>	variable locale de vue
\${sessionScope.role}	lire un attribut de session
\${pageContext.request.contextPath}	préfixer les liens par /gestion-agences

4.4 Les trois graphiques du tableau de bord

Le servlet ne fabrique pas de JSON : il pose des **chaînes déjà formatées**, que la JSP insère dans le script.

```
// HomeServlet
request.setAttribute("chartPointageLabels", "'Presents','Retards','Absents'");
request.setAttribute("chartPointageData", presentsStricts + "," + retards + "," + absents);

new ApexCharts(document.querySelector("#pointageChart"), {
  chart: { type: 'donut', height: 320, background: 'transparent' },
  labels: [{chartPointageLabels}],
  series: [{chartPointageData}]
}).render();
```

C'est simple et sans dépendance de sérialisation. La limite est qu'un libellé contenant une apostrophe casserait le script : pour des données saisies par l'utilisateur, il faudrait passer par du JSON échappé.

5 Le back en détail

5.1 Le motif commun à tous les servlets

Les dix-huit servlets suivent la même structure, ce qui rend le code prévisible :

1. récupérer la session avec `getSession(false)` ;
2. contrôler l'authentification, puis le rôle ;
3. lire les paramètres de la requête ;
4. appeler un ou plusieurs services ;
5. `setAttribute` puis `forward` pour afficher, ou `sendRedirect` après une écriture.

5.2 Les EJB

Quinze services sont `@Stateless`, un seul est `@Singleton`.

<code>@Stateless</code>	Le conteneur gère un réservoir d'instances et en prête une le temps de l'appel. Aucun état conservé entre deux requêtes, donc pas de problème de concurrence.
<code>@Singleton</code>	Une seule instance pour toute l'application. Utilisé pour <code>SecurityService</code> , avec <code>@Startup</code> pour qu'elle existe dès le démarrage.

L'injection est déclarative — aucun `new` sur un service :

```
@EJB
private DemandeModificationService demandeModificationService;
```

5.3 JPA et les transactions

Chaque service reçoit son `EntityManager` par injection :

```
@PersistenceContext(unitName = "default")
private EntityManager em;
```

Les requêtes sont écrites en **JPQL** — sur les entités, pas sur les tables — et **toujours paramétrées** :

```
return em.createQuery(
    "SELECT d FROM DemandeModification d WHERE d.utilisateur = :u "
    + "ORDER BY d.dateDemande DESC", DemandeModification.class)
    .setParameter("u", user)
    .getResultList();
```

Question quasi certaine : et l'injection SQL ? Aucune requête n'est construite par concaténation de chaînes ; les valeurs passent par `setParameter`, donc par une variable liée côté pilote. La valeur n'est jamais interprétée comme du SQL.

Les transactions ne sont pas écrites. L'unité de persistance est déclarée `transaction-type="JTA"` et les services sont des EJB : le conteneur ouvre une transaction à l'entrée de chaque méthode et la valide à la sortie, ou l'annule si une exception s'échappe. C'est la gestion *par le conteneur* (CMT). D'où, dans la trace d'erreur de connexion, la présence de `CMTTxInterceptor` : c'est l'intercepteur qui pose la transaction.

5.4 L'unité de persistance

```
<persistence-unit name="default" transaction-type="JTA">
  <jta-data-source>java:/OracleDS</jta-data-source>
  ... 16 classes d'entites ...
  <properties>
    <property name="hibernate.dialect" value="org.hibernate.dialect.OracleDialect"/>
    <property name="hibernate.hbm2ddl.auto" value="none"/>
  </properties>
</persistence-unit>
```

Deux points méritent d'être signalés au jury :

- `hbm2ddl.auto = none` : Hibernate ne crée ni ne modifie **jamais** le schéma. Les tables sont créées par les scripts de `sql/`, et la base fait autorité.
- aucun identifiant de connexion n'est écrit ici. Ils appartiennent à la datasource `java:/OracleDS`, configurée dans WildFly. Le même WAR se déploie donc sur plusieurs environnements sans être recompilé.

6 Le volet décisionnel

Le volet décisionnel part des **mêmes tables** que l'application. Rien n'est saisi deux fois : trois chemins de restitution s'appuient sur une même base opérationnelle.

Tableau de bord embarqué	Servlets et services de l'application, graphiques ApexCharts — temps réel, dans la page
Power BI	Vues <code>V_BI_*</code> lues directement par Power BI Desktop — rapport de quatre pages
Segmentation	Datamart produit par la chaîne ETL, puis apprentissage non supervisé en Python

6.1 Les vues décisionnelles

Quatre vues forment la **couche sémantique** : elles traduisent les colonnes techniques en notions métier, de sorte que Power BI n'ait ni jointure ni calcul compliqué à refaire.

Vue	Fichier	Contenu
<code>V_BI_EMPLOYES</code>	<code>v_bi_employees.sql</code>	effectifs, rôle calculé, statut
<code>V_BI_CLIENTS</code>	<code>v_bi_clients.sql</code>	portefeuille : identité, sexe, type, statut
<code>V_BI_OBJECTIFS</code>	<code>v_bi_objectifs.sql</code>	production : comptes, crédits, cartes, épargne
<code>V_BI_POINTAGE</code>	<code>setup_pointage.sql</code>	présence : indicateurs présent / retard / absent

`V_BI_OBJECTIFS` illustre le principe : les colonnes brutes de `B_OBJECTIF` sont consolidées et renommées en indicateurs, et les clés sont remplacées par des libellés.

```
CREATE OR REPLACE VIEW V_BI_OBJECTIFS AS
SELECT
    O.ID_OBJECTIF, O.ANNEE_MOIS,
    A.LIBELLE_AGENCE AS NOM_AGENCE,
    U.LIBELLE_UTILISATEUR AS NOM_GESTIONNAIRE,
    O.SITUATION_ADMINISTRATIVE,
    (NVL(O.SOUSCRIPTION_COMPTE_CHEQUES_OA, 0)
+ NVL(O.SOUSCRIPTION_COMPTE_EPARGNES_OA, 0)
+ NVL(O.SOUSCRIPTION_COMPTE_COURANTS_OA, 0)) AS TOTAL_OUVERTURE_COMPTES,
    NVL(O.PRODUCTION_CREDITS_CONSO_OA, 0) AS CREDITS_CONSO,
    NVL(O.PRODUCTION_CREDITS_IMMO_OA, 0) AS CREDITS_IMMO,
    NVL(O.PRODUCTION_CREDITS_INVESTISSEMENT_OA, 0) AS CREDITS_INVEST,
    NVL(O.SOUSCRIPTION_CARTES_UNITE_OA, 0) AS CARTES_COMMANDES,
    NVL(O.EPARGNE_ADD_OA, 0) AS COLLECTE_EPARGNE
FROM B_OBJECTIF O
LEFT JOIN AGENCE A ON O.SK_AGENCE = A.SK_AGENCE
LEFT JOIN B_UTILISATEURS U ON O.SK_UTILISATEUR = U.SK_UTILISATEUR;
```

Le `NVL(..., 0)` systématique et les jointures `LEFT JOIN` sont volontaires : une agence sans gestionnaire renseigné, ou un indicateur non saisi, ne doit pas faire disparaître la ligne du rapport. C'est la première règle d'une couche sémantique : **ne jamais perdre de lignes en chemin**.

6.2 Le schéma BTK_BI

Power BI ne se connecte pas au schéma applicatif. Le script `powerbi/creer_schema_bi.sql` crée un utilisateur et un schéma dédiés, puis y recopie les tables nécessaires :

```
CREATE USER BTK_BI IDENTIFIED BY "...";
GRANT CONNECT, RESOURCE TO BTK_BI;
```

```
ALTER USER BTK_BI QUOTA UNLIMITED ON USERS;

CREATE TABLE BTK_BI.AGENCE AS SELECT * FROM SYSTEM.AGENCE;
CREATE TABLE BTK_BI.B_UTILISATEURS AS SELECT * FROM SYSTEM.B_UTILISATEURS;
CREATE TABLE BTK_BI.B_CLIENTS AS SELECT * FROM SYSTEM.B_CLIENTS;
CREATE TABLE BTK_BI.B_OBJECTIF AS SELECT * FROM SYSTEM.B_OBJECTIF;
CREATE TABLE BTK_BI.POINTAGE AS SELECT * FROM SYSTEM.POINTAGE;
CREATE TABLE BTK_BI.GESTIONNAIRE AS SELECT * FROM SYSTEM.GESTIONNAIRE;
```

Pourquoi séparer ? Un compte de restitution n'a pas à pouvoir écrire dans les tables de production, et les requêtes d'un rapport ne doivent pas peser sur l'application. C'est la même logique qui a mené à la chaîne ETL.

6.3 La chaîne ETL

etl/etl_agences.py consolide les données opérationnelles en un **datamart en étoile agrégé par agence**. Elle enchaîne cinq étapes :

Extract	lit AGENCE, B_UTILISATEURS, CLIENT_BTK, B_OBJECTIF, POINTAGE — par Oracle, par export CSV, ou depuis l'extrait agrégé livré avec le projet
Nettoyage	écarte les lignes sans SK_AGENCE, type les colonnes
Transformation	agrège par agence : effectif, gestionnaires, clients, comptes, crédits, épargne, taux de présence
Contrôle	valeurs manquantes, valeurs négatives, agences en double — le chargement est interrompu si un contrôle échoue
Load	écrit dim_agence, fait_agence, et le fichier consommé par la segmentation

Le cœur de la transformation tient en quelques lignes de pandas :

```
eff = employees.groupby("SK_AGENCE").agg(
    effectif=("SK_UTILISATEUR", "count"),
    nb_gestionnaires=("EST_GESTIONNAIRE", "sum")).reset_index()
cli = clients.groupby("SK_AGENCE").size().reset_index(name="nb_clients")
obj = objectifs.groupby("SK_AGENCE").agg(
    total_comptes=("COMPTES", "sum"),
    production_credits=("CREDITS", "sum"),
    collecte_epargne=("EPARGNE", "sum")).reset_index()

datamart = (agences.merge(eff, on="SK_AGENCE", how="left")
            .merge(cli, on="SK_AGENCE", how="left")
            .merge(obj, on="SK_AGENCE", how="left"))
```

Le résultat porte sur les **49 entités** du réseau extraites de la base. Deux notebooks Jupyter rejouent la chaîne pas à pas (etl/etl_btk.ipynb) et la segmentation (clustering/segmentation_btk.ipynb); chacun se termine par une cellule qui relance le script correspondant et **vérifie que les résultats coïncident**.

6.4 Power BI — la construction du rapport

6.4.1 Connexion

Obtenir les données → **Oracle** → serveur localhost:1521/FREEPDB1, compte BTK_BI. Les vues V_BI_EMPLOYES, V_BI_CLIENTS, V_BI_OBJECTIFS et V_BI_POINTAGE sont importées.

6.4.2 Thème et gabarit

Le dossier powerbi/ fournit btk_theme.json (couleurs, polices, styles des visuels, fond de visuel #212329) et template_background.png, l'image de fond du canevas en **1280×720**. Le thème s'applique par Affichage → Thèmes, le fond par Format de la page → Arrière-plan du canevas.

6.4.3 Les quatre pages

1. Résumé	quatre cartes KPI, deux anneaux (rôles, présence), deux histogrammes (évolution des crédits, production par type), tableau des cinq premières agences
2. Clients	répartition du portefeuille : gouvernorat, secteur d'activité, type de client
3. Performance commerciale	comptes, crédits par nature, épargne, packs et cartes
4. Présence / RH	présents, retards, absences, taux de présence

Des segments communs — District, ANNEE_MOIS, TYPE_CLIENT — filtrent l'ensemble, avec un bouton de réinitialisation. La barre latérale du fond sert de navigation entre les quatre pages.

6.4.4 Les mesures DAX

Les indicateurs sont des **mesures**, calculées au moment de l'affichage et donc sensibles aux filtres. Les plus représentatives :

```
Nb Employes = DISTINCTCOUNT(B_UTILISATEURS[SK_UTILISATEUR])

Total Comptes =
    SUM(B_OBJECTIF[SOUSCRIPTION_COMPTE_CHEQUES_OA])
    + SUM(B_OBJECTIF[SOUSCRIPTION_COMPTE_EPARGNES_OA])
    + SUM(B_OBJECTIF[SOUSCRIPTION_COMPTE_COURANTS_OA])

Total Credits =
    SUM(B_OBJECTIF[PRODUCTION_CREDITS_CONSO_OA])
    + SUM(B_OBJECTIF[PRODUCTION_CREDITS_IMMO_OA])
    + SUM(B_OBJECTIF[PRODUCTION_CREDITS_INVESTISSEMENT_OA])

Taux Presence =
DIVIDE(
    CALCULATE(COUNTROWS(POINTAGE), POINTAGE[STATUT] IN {"PRESENT", "RETARD"}),
    COUNTROWS(POINTAGE)
)
```

Le rôle, lui, est une **colonne calculée** et non une mesure : il qualifie chaque ligne d'employé, il doit donc être matérialisé pour servir d'axe d'analyse.

```
Role =
SWITCH(
    TRUE(),
    B_UTILISATEURS[EST_GESTIONNAIRE] = 1, "Gestionnaire",
    B_UTILISATEURS[EST_GESTIONNAIRE] = 0 && B_UTILISATEURS[SK_AGENCE] = 31, "Hors Commercial",
    "Conseiller"
)
```

Mesure ou colonne calculée ? C'est la question DAX la plus fréquente. Une **colonne calculée** est évaluée une fois, à l'actualisation, et occupe de la mémoire : elle sert quand on veut *filtrer ou grouper* par sa valeur. Une **mesure** est évaluée à l'affichage, dans le contexte des filtres actifs : elle sert à *agréger*. Ici, Role est une colonne (on groupe par rôle), Taux Présence une mesure (il change selon le mois ou l'agence sélectionnés).

6.5 La segmentation des agences

Le datamart alimente un apprentissage non supervisé (clustering/segmentation_reelle.py). Le siège est écarté — 540 employés, il écraserait toute autre structure — et les montants passent au logarithme avant standardisation, leur distribution étant très asymétrique.

Quatre modèles sont comparés sur la **totalité** des agences, les points qu'un modèle refuse de classer étant comptés comme un groupe à part entière, pour qu'aucun ne soit avantagé par le fait d'avoir écarté les cas difficiles.

Modèle	Clusters	Silhouette	Lecture
K-Means	3	0,604	retenu
DBSCAN	4	0,517	rejette CENTRALE comme bruit
GMM	3	0,446	
Agglomératif	3	0,417	

Trois segments interprétables en sortent : 34 agences d'exploitation, 6 entités de production sans porte-feuille, 8 entités sans activité enregistrée.

7 Construction et déploiement

7.1 De la source au WAR

```
./mvnw clean package # Windows : mvnw.cmd clean package
```

Maven compile les sources avec le **JDK 21**, copie les ressources (`persistence.xml`) et les pages web, puis produit `target/gestion-agences.war`. Le *wrapper* (`mvnw`) est versionné avec le projet : il télécharge lui-même la bonne version de Maven, donc la construction ne dépend pas de ce qui est installé sur le poste.

7.2 Ce que contient le WAR

Chemin dans l'archive	Contenu
<code>index.jsp</code> , <code>forgot-password.jsp</code>	pages accessibles sans authentification
<code>css/style.css</code>	feuille de style
<code>WEB-INF/*.jsp</code>	vues protégées, inaccessibles par URL directe
<code>WEB-INF/web.xml</code>	fichier de déploiement (page d'accueil)
<code>WEB-INF/jboss-web.xml</code>	racine de contexte <code>/gestion-agences</code>
<code>WEB-INF/classes/</code>	classes compilées et <code>META-INF/persistence.xml</code>

Le fichier `web.xml` est volontairement minimal : les servlets et le filtre sont déclarés par **annotations** (`@WebServlet`, `@WebFilter`), pas dans le descripteur.

```
<jboss-web>
  <context-root>/gestion-agences</context-root>
</jboss-web>
```

C'est ce fichier qui explique que l'application réponde sur `http://localhost:8080/gestion-agences/` et non sous le nom de l'archive.

7.3 Déployer sur WildFly

Copier le WAR dans `$WILDFLY_HOME/standalone/deployments/`, ou déployer depuis l'IDE — **mais pas les deux** : un WAR copié à la main en plus du déploiement de l'IDE provoque l'erreur `WFLYSRV0205 Duplicate deployment`. Le script `tools/clean-wildfly-deployment.sh` nettoie ce cas.

7.4 La datasource

L'application n'ouvre **jamais** de connexion elle-même : elle demande au serveur la ressource `java:/OracleDS`. Cette datasource est déclarée dans WildFly, avec l'URL, le compte et le mot de passe.

Nom JNDI	java:/OracleDS
URL	jdbc:oracle:thin:@localhost:1521/FREEPDB1
Compte	celui déclaré côté serveur
Pilote	ojdbc11 — le plus simple est de déposer le .jar dans standalone/deployments/

Pourquoi les identifiants ne sont-ils pas dans le projet ? Parce que le même WAR doit pouvoir être déployé en développement, en recette et en production sans être recompilé. Ce qui change d'un environnement à l'autre appartient au serveur, pas à l'archive. C'est aussi ce qui évite de versionner un mot de passe.

7.5 Diagnostiquer une erreur de connexion

L'erreur la plus probable le jour de la soutenance :

```
jakarta.ejb.EJBException: org.hibernate.exception.GenericJDBCException:
Unable to acquire JDBC Connection
[jakarta.resource.ResourceException: IJ000453: Unable to get managed
connection for java:/OracleDS]
```

Elle ne vient **pas** du code : la datasource est bien trouvée, mais son pool n'arrive pas à joindre Oracle. Une commande tranche entre les deux familles de causes :

```
python3 etl/test_connexion_oracle.py
```

Le test échoue	Oracle n'est pas joignable : service arrêté, ou base pluggable restée en MOUNTED après un redémarrage — ALTER PLUGGABLE DATABASE FREEPDB1 OPEN; puis SAVE STATE
Le test réussit	Le problème est la datasource : compte, mot de passe ou nom de service. Corriger par jboss-cli, puis reload

La marche à suivre complète est dans `tools/DEPANNAGE_ORACLE_WILDFLY.md`, et le script de diagnostic dans `tools/setup-oracle-datasource.cli`.

7.6 L'ordre de démarrage, avant la soutenance

1. démarrer Oracle (services Windows) et vérifier que la base pluggable est READ WRITE;
2. démarrer WildFly;
3. ouvrir `http://localhost:8080/gestion-agences/`;
4. si une page a été ouverte pendant qu'Oracle était absent, vider le pool : `flush-all-connection-in-pool`.

8 Les questions probables, et quoi répondre

8.1 Architecture

« Pourquoi des servlets et des JSP, et pas Spring Boot ? »

Le projet reste sur la pile **Jakarta EE standard** : les servlets, les EJB et JPA font partie de la plateforme, et WildFly les fournit. Il n'y a donc aucune dépendance de *framework* à gérer dans le `pom.xml`, et l'application se déploie sur n'importe quel serveur compatible. Spring Boot aurait apporté de la configuration automatique, au prix d'une pile supplémentaire à maîtriser et à justifier.

« Qu'est-ce qui garantit la séparation des couches ? »

Trois choses vérifiables dans le code : aucun servlet n'injecte d'`EntityManager` — ils n'ont que des `@EJB`; aucun service ne manipule `HttpServletRequest`; aucune JSP ne contient de scriptlet Java. Chaque couche ne connaît que la suivante.

« Pourquoi les JSP sont-elles sous WEB-INF ? »

Parce que le conteneur en interdit l'accès par URL directe. On ne peut y entrer que par un forward depuis un servlet, donc après le contrôle de session et de rôle. Les deux exceptions, `index.jsp` et `forgot-password.jsp`, sont volontaires : elles doivent être atteignables sans être connecté.

8.2 Sécurité

« Où est contrôlé le rôle ? »

Au début de `doGet` et `doPost` de chaque servlet concerné. Le menu masque aussi les entrées interdites, mais ce n'est **qu'un confort** : la vue est contournable, le contrôle serveur ne l'est pas.

« Et si quelqu'un forge une requête HTTP ? »

Sur le circuit de validation, trois barrières se superposent : la vue n'affiche le bouton que pour le bon rôle et le bon état ; le servlet croise l'action et le rôle et rejette toute combinaison non prévue ; le service vérifie l'état de départ de la demande avant d'agir. Les deux dernières sont côté serveur.

« Comment vous protégez-vous de l'injection SQL ? »

Aucune requête n'est construite par concaténation. Tout passe par JPQL avec `setParameter`, donc par des variables liées : la valeur n'est jamais interprétée comme du code SQL.

« Le mot de passe est-il protégé ? » — à annoncer soi-même plutôt que de le subir

Non : il est comparé en clair dans `SecurityService.authenticate`. C'est la limite principale du projet. La correction est localisée : stocker un condensat salé (BCrypt, Argon2), charger l'utilisateur par son seul identifiant, puis comparer en Java. Une seule méthode et une migration de la colonne.

8.3 JPA et transactions

« Qui ouvre et ferme les transactions ? »

Le conteneur. L'unité de persistance est en `transaction-type="JTA"` et les services sont des EJB : une transaction est ouverte à l'entrée de chaque méthode et validée à la sortie, ou annulée si une exception s'échappe. C'est la raison de la présence de `CMTTxInterceptor` dans les traces.

« Pourquoi `hbm2ddl.auto = none` ? »

Pour qu'Hibernate ne touche jamais au schéma. Les tables sont créées par les scripts de `sql/`, et la base fait autorité. Un update automatique en production pourrait modifier une structure existante sans contrôle.

« Différence entre `persist` et `merge` ? »

`persist` enregistre une entité nouvelle ; `merge` réintègre une entité détachée et renvoie l'instance gérée. Le projet applique le motif classique : `if (entite.getId() == null) persist else merge`.

8.4 Front

« Pourquoi pas de framework front ? »

Les pages sont des listes et des formulaires, rendus par le serveur : un client riche n'apporterait rien et ajouterait une étape de construction, un gestionnaire de paquets et des dépendances à suivre. Deux bibliothèques suffisent : `ApexCharts` pour les graphiques, `Lucide` pour les icônes.

« Comment les données arrivent-elles dans les graphiques ? »

Le servlet pose des chaînes déjà formatées avec `setAttribute`, la JSP les insère dans la configuration `ApexCharts`. Pas de sérialisation JSON ni d'appel asynchrone. La limite est qu'un libellé contenant une apostrophe casserait le script : pour des données saisies par l'utilisateur, il faudrait du JSON échappé.

8.5 Décisionnel

« Pourquoi des vues `V_BI_*` plutôt que les tables ? »

Pour poser une **couche sémantique** : les colonnes techniques deviennent des indicateurs métier, les clés deviennent des libellés, les valeurs nulles sont ramenées à zéro et les jointures sont faites une fois pour toutes. Power BI consomme du sens, pas de la structure. Et si le schéma évolue, seule la vue change, pas le rapport.

« Pourquoi une chaîne ETL en plus de Power BI ? »

Parce que les deux ne servent pas au même usage. Power BI restitue au niveau du détail ; la segmentation a besoin d'un **datamart agrégé par agence**, avec un contrôle de qualité et une sortie stable, reproductible d'une exécution à l'autre. L'ETL produit ce jeu, et le clustering le consomme.

« Mesure ou colonne calculée ? »

Une colonne calculée est évaluée à l'actualisation et occupe de la mémoire : elle sert à *filtrer ou grouper*. Une mesure est évaluée à l'affichage, dans le contexte des filtres actifs : elle sert à *agréger*. Dans le rapport, Rôle est une colonne, Taux Présence une mesure.

« Pourquoi K-Means et pas un autre modèle ? »

Parce que c'est celui qui obtient le meilleur score de silhouette (0,604) **sous un protocole équitable** : les quatre modèles sont notés sur la totalité des agences, et les points que DBSCAN refuse de classer sont comptés comme un groupe. Sans cette précaution, un modèle serait avantage pour avoir écarté les cas difficiles.

« Pourquoi écarter le siège de la segmentation ? »

540 employés contre une dizaine pour une agence : c'est une structure centrale, pas une agence. Le laisser reviendrait à créer un groupe d'un seul point et à écraser les distances entre toutes les autres. Il reste dans le datamart, il est seulement retiré de l'analyse.

9 Points ouverts et limites

Les annoncer soi-même vaut mieux que de les subir. Ils ont été relevés par un inventaire du code fait pour ce dossier.

Mot de passe en clair	Comparé tel quel dans <code>SecurityService.authenticate</code> . Correction : condensat salé, comparaison en Java.
Contrôle de rôle partiel	Quatre servlets ne vérifient que l'authentification, sans le rôle : <code>BUtilisateurServlet (/employes)</code> , <code>BObjectifServlet (/objectifs-btk)</code> , <code>ObjectifServlet (/objectifs)</code> et <code>ClientServlet</code> . Ce sont des écrans de consultation, mais l'application est donc plus permissive que la matrice des droits.
Page gestionnaires	<code>GestionnaireServlet</code> transfère vers <code>/WEB-INF/gestionnaires.jsp</code> alors que le fichier s'appelle <code>gestionnaire.jsp</code> : la page échoue. Elle n'est pas dans le menu. Correction : un mot dans le chemin du forward.
Reliquat historique	<code>ClientServlet (/clients-old)</code> et <code>client.jsp</code> ne sont atteignables par aucun lien du menu ; <code>client.jsp</code> pointe vers <code>/clients</code> , qui n'est associé à aucun servlet.
Pas de tests automatisés	Le projet n'a pas de tests unitaires. Les services étant des EJB sans état, ils s'y prêteraient bien (JUnit + base en mémoire).
Graphiques et échappement	Les séries ApexCharts sont injectées comme chaînes ; un libellé contenant une apostrophe casserait le script.

Comment présenter une limite. La formuler, en donner la cause, puis la correction et son coût. « Le mot de passe est comparé en clair ; c'est un choix de périmètre, pas un oubli ; la correction tient dans une méthode et une migration de colonne. » Une limite reconnue et chiffrée est un signe de maîtrise ; une limite découverte par le jury ne l'est pas.

10 Récapitulatif : la carte des URL

URL	Servlet	Rôles	Vue
/login	LoginServlet	ouverte	index.jsp
/logout	LogoutServlet	connecté	—
/forgot-password	ForgotPasswordServlet	ouverte	forgot-password.jsp
/home	HomeServlet	connecté	home.jsp
/employes	BUtilisateurServlet	connecté	employes.jsp

URL	Servlet	Rôles	Vue
/clients-btk	BClientServlet	ADMIN	clients-btk.jsp
/banques	BanqueServlet	ADMIN	banques.jsp
/agences	AgenceServlet	ADMIN	agences.jsp
/objectifs-btk	BObjectifServlet	connecté	objectifs-btk.jsp
/objectifs	ObjectifServlet	connecté	objectifs.jsp
/pointage	PointageServlet	ADMIN (saisie manuelle)	pointage.jsp
/ajouter-employe	AjouterEmployeServlet	ADMIN	add-effectif.jsp
/demandes-agence	DemandeAgenceServlet	ADMIN, DIRECTEUR	demandes-agence.jsp
/demandes-admin	DemandeAgenceAdminServlet	ADMIN	demandes-admin.jsp
/gestion-demandes	GestionModificationsServlet	ADMIN, DIRECTEUR	gestion-demandes.jsp
/mes-demandes	MesDemandesServlet	connecté	profil.jsp
/gestionnaires	GestionnaireServlet	ADMIN	<i>transfert erroné</i>
/clients-old	ClientServlet	connecté	<i>reliquat</i>